

Because we are doing this check in every frame, the movement of the ball will appear to be continuous as long as the user holds down a key. For example, if the user presses and holds the right arrow key, this code will add 3 to the value of the `ballX` coordinate in every frame, and the user will see the ball moving smoothly to the right. When they stop pressing the key, the movement stops.

The other change is that this approach allows you to check for multiple keys being down at the same time. For example, if the user presses and holds the left and down arrow keys, the ball will move diagonally down and to the left. You can check for as many keys being held down as you wish. However, the number of *simultaneous* key presses that can be detected is limited by the operating system, the keyboard hardware, and many other factors. The typical limit is around four keys, but your mileage may vary.

Creating a Location-Based Animation

Next, we'll build a location-based animation. This code will allow us to move an image diagonally and then have it appear to bounce off the edges of the window. This was a favorite technique of screensavers on old CRT-based monitors, to avoid burning in a static image.

We'll change the location of our image slightly in every frame. We'll also check if the result of that movement would place any part of the image outside one of the window boundaries and, if so, reverse the movement in that direction. For example, if the image was moving down and would cross the bottom of the window, we would reverse the direction and make the image start moving up.

We'll again use the same starting template. [Listing 5-6](#) gives the full source code.

File:

PygameDemo4_OneBallBounce/PygameOneBallBounceXY.py

```
# pygame demo 4(a) - one image, bounce around the window using (x, y) coords

# 1 - Import packages
import pygame
from pygame.locals import *
```